

Monster Hunter Loadout Manager (Backend)

Roman Truninger

Inhaltsverzeichnis

Inhaltsverzeichnis	2
Projektidee	3
Anforderungskatalog	4
Klassendiagramm	6
Ablauf je User Story	7
Datentyp je Endpoint	15
Testplan	24
Testplan Durchführung	32
Installationsanleitung	33
Hilfestellungen & Quellen	34

Projektidee

Das Backend für eine Webapp zu bauen.

Das Backend implementiert volles CRUD über ArmorLoadouts von dem Videospiel Monster Hunter World.

Zusätzlich gibt es natürlich die Möglichkeit alle ArmorPieces die es gibt mit Read auszulesen (wie sollte man sonst ein ArmorLoadout aus validen Rüstungsteilen aka ArmorPieces zusammenstellen?)

Diese API / Backend ist sehr praktisch, wenn man eine Übersicht über die verschiedenen Rüstungsteile und Rüstungssets, die man aus denen Rüstungsteilen zusammenstellen kann, behalten will.

Anforderungskatalog

User Story 1

Als Benutzer möchte ich ein neues Loadout speichern können, damit ich meine Lieblingsloadouts nicht vergessen werde.

Akzeptanzkriterien:

- Get localhost:8080/api/armorpieces Returniert eine Liste von allen ArmorPieces
- Wenn man dann ein Post localhost:8080/api/armorloadouts mit einem namen und validen armorpieces sendet - sollte man Response 201 bekommen

User Story 2

Als Spieler möchte ich ein schon bestehendes Loadout aktualisieren können, damit meine Loadouts jederzeit veränderbar bleiben.

Akzeptanzkriterien:

- Json von Get localhost:8080/api/armorloadouts/{id von loadout das man verändern will} Kopieren, feld id löschen
- Den kopierten inhalt mit der geplanten veränderung ändern
- Wenn man jetzt Put localhost:8080/api/armorloadouts/{id von loadout das man verändern will} mit dem geänderten kopierten inhalt sendet, sollte man Response Code 200 zurück bekommen

User Story 3

Als Spieler möchte ich den Inhalt von schon bestehenden Loadouts anzeigen können, damit ich Einsicht haben kann, welche Rüstungsteile in den jeweiligen Loadouts gespeichert sind.

Akzeptanzkriterien:

- Wenn man `Get localhost:8080/api/armorloadouts` sendet, sollte man das json von allen bestehenden armorloadouts sehen, und den Response Code 200 bekommen

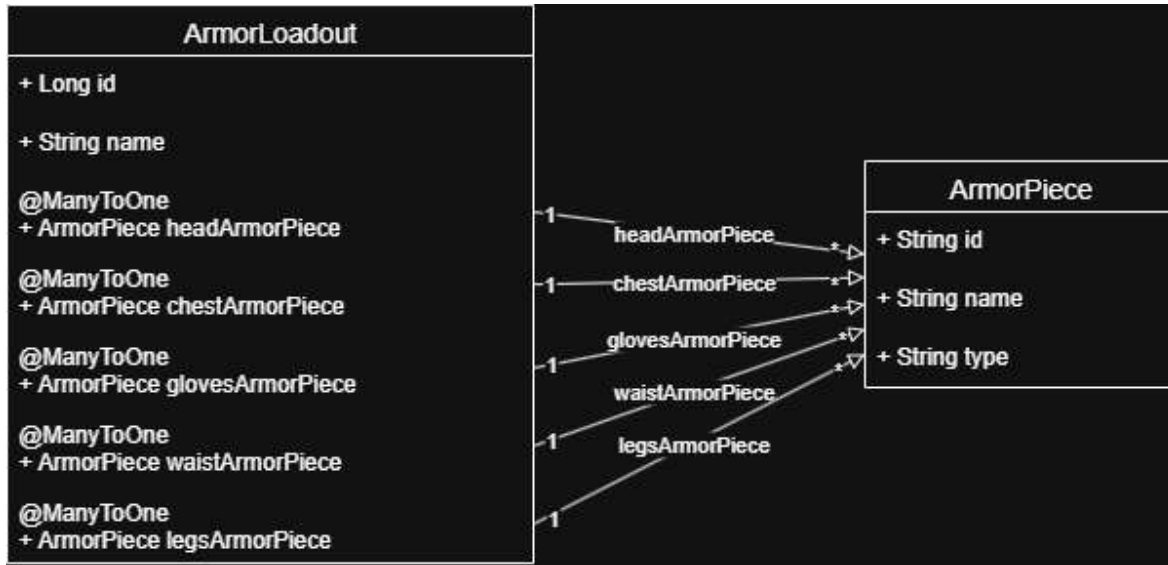
User Story 4

Als Spieler möchte ich ein schon bestehendes Loadout löschen können, damit ich die Anzahl gespeicherten Loadouts übersichtlich behalten kann.

Akzeptanzkriterien:

- Wenn man `Delete localhost:8080/api/armorloadouts/{id von dem Loadout, welches man löschen will}` sendet, sollte das ArmorLoadout gelöscht worden sein, und man den Response Code 204 bekommen

Klassendiagramm



Ablauf je User Story

User Story 1

Als Benutzer möchte ich ein neues Loadout speichern können, damit ich meine Lieblingsloadouts nicht vergessen werde.

GET localhost:8080/api/armorpieces

Response: Code 200

```
[
  {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  ...
]
```

=> **POST** localhost:8080/api/armorloadouts

Request body:

```
{
  "name": "exampleNameOfLoadout",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  }
}
```


Response: Code 201

```
{
  "name": "exampleNameOfLoadout",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```

User Story 2

Als Spieler möchte ich ein schon bestehendes Loadout aktualisieren können, damit meine Loadouts jederzeit veränderbar bleiben.

GET localhost:8080/api/armorloadouts/51

Response:

```
{
  "name": "exampleNameOfLoadout",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```

=> **PUT** localhost:8080/api/armorloadouts/51

Request body:

```
{
  "name": "changedName",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  }
}
```

Response: Code 200

```
{
  "name": "changedName",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```

User Story 3

Als Spieler möchte ich den Inhalt von schon bestehenden Loadouts anzeigen können, damit ich Einsicht haben kann, welche Rüstungsteile in den jeweiligen Loadouts gespeichert sind.

GET localhost:8080/api/armorloadouts

Response: Code 200

```
[
  {
    "name": "changedName",
    "headArmorPiece": {
      "id": "1",
      "name": "Leather Headgear",
      "type": "head"
    },
    "chestArmorPiece": {
      "id": "2",
      "name": "Leather Mail",
      "type": "chest"
    },
    "glovesArmorPiece": {
      "id": "3",
      "name": "Leather Gloves",
      "type": "gloves"
    },
    "waistArmorPiece": {
      "id": "4",
      "name": "Leather Belt",
      "type": "waist"
    },
    "legsArmorPiece": {
      "id": "5",
      "name": "Leather Trousers",
      "type": "legs"
    },
    "id": 1
  }
]
```

User Story 4

Als Spieler möchte ich ein schon bestehendes Loadout löschen können, damit ich die Anzahl gespeicherten Loadouts übersichtlich behalten kann.

GET localhost:8080/api/armorloadouts/51

Response: Code 204

```
connection: keep-alive
date: Sat, 11 Jul 2026 10:29:37 GMT
keep-alive: timeout=60
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
```

Datentyp je Endpoint

ArmorLoadoutController

GET /api/armorloadouts

Request: GET localhost:8080/api/armorloadouts

Response: Code 200

```
[
  {
    "name": "name",
    "headArmorPiece": {
      "id": "1",
      "name": "Leather Headgear",
      "type": "head"
    },
    "chestArmorPiece": {
      "id": "2",
      "name": "Leather Mail",
      "type": "chest"
    },
    "glovesArmorPiece": {
      "id": "3",
      "name": "Leather Gloves",
      "type": "gloves"
    },
    "waistArmorPiece": {
      "id": "4",
      "name": "Leather Belt",
      "type": "waist"
    },
    "legsArmorPiece": {
      "id": "5",
      "name": "Leather Trousers",
      "type": "legs"
    },
    "id": 1
  }
]
```

GET /api/armorloadouts/{id}

Request: GET localhost:8080/api/armorloadouts/1

Response: Code 200

```
{
  "name": "name",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```


PUT /api/armorloadouts/{id}

Request: **PUT** localhost:8080/api/armorloadouts/1

```
{
  "name": "changedName",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  }
}
```

Response: Code 200

```
{
  "name": "changedName",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```

POST /api/armorloadouts

Request: **POST** localhost:8080/api/armorloadouts

```
{
  "name": "name",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  }
}
```

Response: Code 201

```
{
  "name": "name",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 54
}
```

DELETE /api/armorloadouts/{id}

Request: **DELETE** localhost:8080/api/armorloadouts/1

Response: Code 204

```
connection: keep-alive
```

```
date: Sat, 11 Jul 2026 10:54:40 GMT
```

```
keep-alive: timeout=60
```

```
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
```

ArmorPieceController

GET /api/armorpieces

Request: GET localhost:8080/api/armorpieces

Response: Code 200

```
[
  {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  ...
]
```

GET /api/armorpieces{id}

Request: **GET** localhost:8080/api/armorpieces/1

Response: Code 200

```
{
  "id": "1",
  "name": "Leather Headgear",
  "type": "head"
}
```

GET /api/armorpieces/type/{type} // head, chest, gloves, waist oder legs

Request: **GET** localhost:8080/api/armorpieces/type/head

Response: Code 200

```
[
  {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  {
    "id": "6",
    "name": "Chainmail Headgear",
    "type": "head"
  },
  {
    "id": "11",
    "name": "Hunter's Headgear",
    "type": "head"
  },
  ...
]
```

Testplan

T1: Inkorrekt zu versuchen ein neues Loadout zu erstellen

- **Vorbedingung:** backend ist am laufen
- **Schritte:**
 1. **POST** localhost:8080/api/armorloadouts
 2. With Request body:

```
{
  "name": "name",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "string",
    "name": "string",
    "type": "string"
  }
}
```

3. Request senden

- **Erwartetes Ergebnis:**

Response: Code 500

```
{  
  "timestamp": "2026-07-11T10:37:41.964Z",  
  "status": 500,  
  "error": "Internal Server Error",  
  "path": "/api/armorloadouts"  
}
```

Response headers:

```
connection: close  
content-type: application/json  
date: Sat, 11 Jul 2026 10:37:41 GMT  
transfer-encoding: chunked  
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
```

T2: Bestehende Loadouts anzeigen

- **Vorbedingung:** backend ist am laufen, mindestens 1 ArmorLoadout ist schon gespeichert
- **Schritte:**
 1. **GET** localhost:8080/api/armorloadouts
 2. senden
- **Erwartetes Ergebnis:**

Response: Code 200

```
[
  {
    "name": "name",
    "headArmorPiece": {
      "id": "1",
      "name": "Leather Headgear",
      "type": "head"
    },
    "chestArmorPiece": {
      "id": "2",
      "name": "Leather Mail",
      "type": "chest"
    },
    "glovesArmorPiece": {
      "id": "3",
      "name": "Leather Gloves",
      "type": "gloves"
    },
    "waistArmorPiece": {
      "id": "4",
      "name": "Leather Belt",
      "type": "waist"
    },
    "legsArmorPiece": {
      "id": "5",
      "name": "Leather Trousers",
      "type": "legs"
    },
    "id": 1
  }
]
```

T3: Neues Loadout erstellen

- **Vorbedingung:** backend ist am laufen
- **Schritte:**
 1. **POST** localhost:8080/api/armorloadouts
 2. With Request body:

```
{
  "name": "name",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  }
}
```

3. Request senden

- Erwartetes Ergebnis:

Response: Code 201

```
{
  "name": "name",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```

T4: Bestehendes Loadout löschen

- **Vorbedingung:** backend ist am laufen, mindestens 1 ArmorLoadout ist schon gespeichert
- **Schritte:**
 1. **DELETE** localhost:8080/api/armorloadouts{1}
 2. Request senden
- **Erwartetes Ergebnis:**

Response: Code 204

```
connection: keep-alive
date: Sat, 11 Jul 2026 10:54:40 GMT
keep-alive: timeout=60
vary: Origin, Access-Control-Request-Method, Access-Control-Request-Headers
```

T5: Bestehendes Loadout ändern

- **Vorbedingung:** backend ist am laufen, mindestens 1 Loadout ist schon gespeichert
- **Schritte:**
 1. **PUT** localhost:8080/api/armorloadouts{1}
 2. With Request body:

```
{
  "name": "changedName",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  }
}
```

3. Request senden

- Erwartetes Ergebnis:

Response: Code 200

```
{
  "name": "changedName",
  "headArmorPiece": {
    "id": "1",
    "name": "Leather Headgear",
    "type": "head"
  },
  "chestArmorPiece": {
    "id": "2",
    "name": "Leather Mail",
    "type": "chest"
  },
  "glovesArmorPiece": {
    "id": "3",
    "name": "Leather Gloves",
    "type": "gloves"
  },
  "waistArmorPiece": {
    "id": "4",
    "name": "Leather Belt",
    "type": "waist"
  },
  "legsArmorPiece": {
    "id": "5",
    "name": "Leather Trousers",
    "type": "legs"
  },
  "id": 1
}
```

Testplan Durchführung

ID	Testfall	Status	Bemerkung
T1	Inkorrekt zu versuchen ein neues Loadout zu erstellen	OK	
T2	Bestehende Loadouts anzeigen	OK	
T3	Neues Loadout erstellen	OK	
T4	Bestehendes Loadout löschen	OK	
T5	Bestehendes Loadout ändern	OK	

Installationsanleitung

1. IntelliJ öffnen
2. Clone Repository>Repository URL>
URL: <https://github.com/aura2062/m295mhlmbackend.git>
3. Directory: (irgend ein leerer Ordner Ihrer Wahl auswählen)
4. Auf **Clone** drücken
5. "Trust and Open Project 'm295mhlmbackend'"?
Drücke auf **Trust Project**
6. "Do you want to allow this app to make changes to your device?"
Drücke auf **Yes**
7. Terminal öffnen
8. Sicherstellen das Docker Desktop installiert und gestartet ist (am besten andere container pausieren, so dass port :8080 definitiv frei ist)
9. `docker compose up -d` ← in root folder des projects ausführen, dort wo die docker-compose.yml file ist
10. Fertig: Die Applikation kann man jetzt ganz normal via IntelliJ starten

Hilfestellungen & Quellen

Ich bin durch die Google Classroom Blöcke 01 bis 07 durchgegangen und habe die Anleitung usw für das Beispielprojekt als grosse Inspiration und Hilfestellung genommen => Teilweise ist gewisser Java Code eigentlich genau das, was im Lernmaterial stand, aber halt auf mein Projekt angepasst.

KI:

- In ArmorLoadout.java habe ich DeepSeek benutzt um zu entscheiden / Brainstormen wie genau ich die Liste von ArmorPieces speichern soll, als ArrayList oder nicht zum Beispiel, sowie Wie ich den Konstruktor designen soll.

- In ArmorLoadout.java habe ich DeepSeek, nachdem ich den Block 07 angeschaut habe. Ich habe es benutzt weil ich Realisiert habe, dass ich das Armorloadout umbauen muss, damit die Verbindung ManyToOne anstatt ManyToMany ist.

Die KI hat mir vorgeschlagen, dass ich ja einfach, dank SpringBoot, einfach anstatt eine ArrayListe Von ArmorPieces, 5 individuelle felder haben kann, da eine ArmorLoadout sowieso genau immer 1 von jedem Rüstungsteilslot haben soll (head, chest, gloves, waist, legs. Siehe Code von ArmorLoadout.java für Details)

Und ja, ich habe mich dann für das Entschieden, da es mein Problem löst, sowie auch wie ich persönlich finde, generell mehr Sinn macht.